

Label-Free Multivariate Time Series Anomaly Detection

Qihang Zhou , *Student Member, IEEE*, Shibo He , *Senior Member, IEEE*, Haoyu Liu ,
Jiming Chen , *Fellow, IEEE*, and Wenchao Meng , *Senior Member, IEEE*

Abstract—Anomaly detection in multivariate time series has been widely studied in one-class classification (OCC) setting. The training samples in this setting are assumed to be normal. In more practical situations, it is difficult to guarantee that all samples are normal. Meanwhile, preparing a completely clean training dataset is costly and laborious. Such a case may degrade the performance of OCC-based anomaly detection methods which fit the training distribution as the normal distribution. To overcome this limitation, in this paper, we propose MTGFlow, an unsupervised anomaly detection approach for Multivariate Time series anomaly detection via dynamic Graph and entity-aware normalizing Flow. MTGFlow first estimates the density of the entire training samples and then identifies anomalous instances based on the density of the test samples within the fitted distribution. This relies on a widely accepted assumption that anomalous instances exhibit more sparse densities than normal ones, with no reliance on the clean training dataset. However, it is intractable to directly estimate the density due to the complex dependencies among entities and their diverse inherent characteristics, not to mention detecting anomalies based on the estimated distribution. In order to address these problems, we utilize the graph structure learning model to learn interdependent and evolving relations among entities, which effectively captures the complex and accurate distribution patterns of multivariate time series. In addition, our approach incorporates the unique characteristics of individual entities by employing an entity-aware normalizing flow. This enables us to represent each entity as a parameterized normal distribution. Furthermore, considering that some entities present similar characteristics, we propose a cluster strategy that capitalizes on the commonalities of entities with similar characteristics, resulting in more precise and detailed density estimation. We refer to this cluster-aware extension as MTGFlow_cluster. Extensive experiments are conducted on six widely used benchmark datasets, in which MTGFlow and MTGFlow_cluster demonstrate their superior detection performance.

Index Terms—Multivariate time series, anomaly detection, unsupervised learning.

I. INTRODUCTION

MULTIVARIATE time series (MTS) are commonly found in various contexts, such as smart factories where data is produced by multiple devices, and smart grids where monitoring data is generated by various sensors [1]. Anomalies in MTS refer to unusual patterns or behaviors that occur at a particular time or over a specific period of time [2], [3], [4], [5]. Previous methods mainly center on developing one-class classification (OCC) models, which rely solely on normal data to detect anomalies [6], [7], [8], [9], [10], [11]. The fundamental assumption of them is that the training dataset with all normal samples can be easily obtained [12]. However, the premise that the training dataset contains all normal samples may not always hold true in real-world applications [13], [14], [15], [16], [17], [18]. This leads to noisy training datasets with a mixture of normal and abnormal data instances. Using these training datasets will result in overfitting of the model to noisy labels [19]. As shown in Fig. 1(a), this degrades the performance of those OCC-based methods [20], [21]. Therefore, it is rewarding to develop unsupervised MTS anomaly detection methods based on the dataset with absolute zero known labels.

Density estimation is a promising approach for unsupervised anomaly detection because they do not depend on the assumption that training datasets are all normal. As presented in Fig. 1(b), it detects an abnormal sample based on the widely accepted hypothesis that abnormal instances exhibit sparse densities than the normal, i.e., abnormal samples are typically in the low-density regions while normal samples are in the high-density regions [22], [23], [24]. Previous works have attempted this strategy for time series prediction and the key challenge lies in the accurate density estimation of the distribution. In [25], [26], [27], time series density is modeled as the parameterized probability distribution, while it is still challenging to model a more complex data distribution [28].

More recently, GANF [29] explores the density estimation to tackle MTS anomaly detection task. In their design, the static directed acyclic graph (DAG) is leveraged to model intractable dependence among multiple entities, and normalizing flow is employed to estimate an overall distribution for all entities together [30], [31]. Although GANF has achieved state-of-the-art (SOTA) results, it may suffer from two drawbacks. First, rather than a static inter-relationship, in real-world applications, the mutual dependencies among entities could be evolving. For instance, in a water treatment plant, when the motorized valve is

Manuscript received 16 April 2023; revised 4 November 2023; accepted 27 December 2023. Date of publication 4 January 2024; date of current version 10 June 2024. This work was supported by the National Natural Science Foundation Program of China under Grants U1909207 and U21B2029. Recommended for acceptance by Y. Tong. (Corresponding author: Shibo He.)

Qihang Zhou, Shibo He, Jiming Chen, and Wenchao Meng are with the State Key Laboratory of Industrial Control Technology, Zhejiang University, Hangzhou, Zhejiang 310027, China (e-mail: zqhang@zju.edu.cn; s18he@zju.edu.cn; cjm@zju.edu.cn; wmengzju@zju.edu.cn).

Haoyu Liu is with the State Key Laboratory of Industrial Control Technology, Zhejiang University, Hangzhou, Zhejiang 310027, China, and also with Fuxi AI Lab, NetEase Games, Hangzhou 310052, China (e-mail: haoyu_liu@zju.edu.cn).

Digital Object Identifier 10.1109/TKDE.2024.3349613

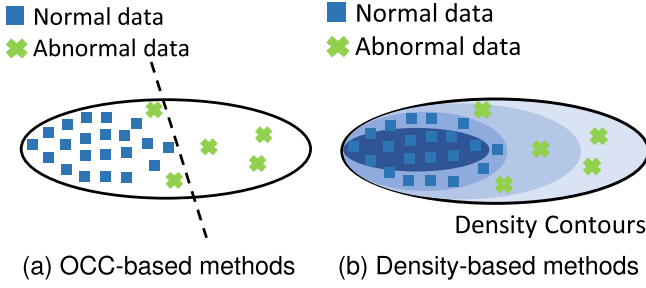


Fig. 1. Schematic diagram of OCC-based and density-based methods on a dataset with mixed normal and abnormal data. *Left*: OCC-based methods are vulnerable to noisy labels, leading to inaccurate decision boundaries. *Right*: Unlike OCC-based methods, density-based methods estimate the density of the test samples, where anomalies are located in low-density regions. Note that the deeper the color, the higher the density.

turned on, it causes the water level to rise. And when the water level exceeds a certain threshold, the motorized valve is closed. These interactions between the actuator of the motorized valve and the water level sensor are strongly coupled. Meanwhile, due to different work conditions, the water level needs to rise for storing water or decrease for draining water. Such relationships are not set in stone, and thus the static graph structure fails to capture this evolving feature. *Question 1: Since dependencies among multi-entities are mutual, is there a better choice to model complicated dependencies than the probabilistic model DAG?* Second, entities usually have diverse working mechanisms, leading to diverse sparse characteristics when anomalies occur. And, GANF projects all entities into the same distribution. *Question 2: Individual anomalous time series from different entities present diverse sparse characteristics. Is it suitable to map all these individual time series to the same latent space (i.e., normal Gaussian distribution)?*

In this paper, we propose MTGFlow and MTGFlow_cluster, unsupervised anomaly detection methods for MTS anomaly detection, to answer the above two questions. First, considering the evolving relations among entities, we introduce graph structure learning to model these changeable interdependencies. To learn the dynamic structure, a self-attention module [32] is plugged into our model for its superior performance on quantifying pairwise interaction. Second, we introduce an entity-aware normalizing flow to handle the diverse inherent characteristics among entities. This approach enables us to model entity-specific density estimation, allowing each entity to be associated with a unique target distribution. Consequently, we can estimate the densities of various entities independently. Moreover, the characteristics of entities are strongly connected to their positions in common scenarios. In industrial IoT, sensors within the same production line or workshop have common characteristics due to their exposure to the same industrial processes and operating conditions [33]. For example, temperature and humidity sensors monitor the same area [3], [34]. Similarly, the common wind speed sensors in wind turbines exhibit similar wind speeds and variations in the same regions [35]. In the domain of transportation, traffic flow sensors along the same stretch of highway or road will exhibit similar patterns because they are influenced by

similar traffic conditions and weather [36]. Identifying the common information makes for fine-grained representation learning for downstream tasks [37], [38]. Motivated by this, we propose to cluster these entities into different groups. Each group is assigned to the same target distribution while also preserving the distinctiveness among different groups. This results in the creation of MTGFlow_cluster. Such a clustering strategy enables us to efficiently handle entities with similar properties and better model their respective density distributions. Incorporating these proposed technologies provides a powerful framework for tackling the complicated characteristics among diverse entities. However, this fine-grained density estimation of MTGFlow and MTGFlow_cluster multiplies memory overhead as the number of entities/clusters increases. We share entity/cluster-specific model parameters to reduce model size. As a result, MTGFlow and MTGFlow_cluster achieve more fine-grained density estimation without extra memory consumption.

Finally, all modules of MTGFlow and MTGFlow_cluster are jointly optimized for overall performance, and maximum log likelihood estimation (MLE) is used to train them. We explore MTGFlow and its extension in various settings including unsupervised and OCC settings. By comparing empirical results of these two settings, MTGFlow and MTGFlow_cluster are found to be robust to anomaly contamination and adapted to more general scenarios. Experiments are conducted on six public datasets to demonstrate the effectiveness of MTGFlow and MTGFlow_cluster, which make progress over the state-of-the-art (SOTA) methods [15], [29], [39], [40], [41], [42], [43], particularly outperforming 5% on SWaT. Besides, we attempt to apply MTGFlow to univariate series (i.e., UCR dataset [44]), and it still attains superior results.

Our contributions are summarized as follows:

- We propose MTGFlow and MTGFlow_cluster for unsupervised MTS anomaly detection. It essentially enables anomaly localization and interpretation without any labels.
- We model the complicated dependencies among entities into the dynamic graph, capturing the complex and evolving mutual dependencies among entities.
- Aiming at different sparse characteristics existing in individual entities, entity-aware normalizing flow is introduced to produce entity-specific density estimation.
- Considering the characteristics of entities are strongly connected to their positions, we propose MTGFlow_cluster, which exploits cluster-aware normalizing flow to capitalize on the commonality of these entities and keep the uniqueness among entities with different characteristics.
- Experiments on six datasets with seven baseline methods demonstrate the superiority of MTGFlow and MTGFlow_cluster, which outperform the SOTA methods.

It is worth mentioning that a conference version of this research was presented in [45]. The conference paper does not involve any priors. To consider the diverse characteristics of entities, MTGFlow maps all entities to unique distributions for anomaly detection. In reality, many entities in industrial or smart grids usually present similar characteristics [46], [47]. For example, water level sensors are used to detect the water level in the same area but in different locations. The cluster information

is readily available, and mapping similar characteristics into the different distributions is not an optimal choice. Therefore, we propose MTGFlow_cluster to perform more fine-grained density estimation, which not only considers the diverse characteristics among entities but also introduces a clustering strategy that capitalizes on the commonalities of entities with similar characteristics. Given the cluster prior, this approach has the potential to further improve anomaly detection performance. Moreover, we supplement the experiments about the OCC setting and univariate time series for the overall evaluation. In addition, more illustrations and visualizations are provided to make a comprehensive analysis of our proposed methods.

II. RELATED WORK

In this section, we introduce recent work in the field of time series anomaly detection, with a particular focus on graph structure learning and normalizing flow techniques.

A. Time Series Anomaly Detection

Anomaly detection for time series is a classical research topic, which has been extensively investigated under OCC setting [48]. Temporal correlation is one of the most important features of time series [49]. Compared with the normal, anomalous time points and sequences often present unusual temporal correlations. To model the distribution of normal time series, DeepSVDD [40] maps training data into preset hypersphere, assuming that anomalous data lie outside this space during the test. EncDecAD [50] leverages LSTM [51] to extract sequence features and designs the reconstruction task to detect anomalies. USAD [43] and DAEMON [8] use adversarial learning to promote reconstruction quality. Considering the much more complex temporal dependence of MTS, OmniAnomaly [7] constructs informative stochastic representations for more robust performance. More recent works [10] and [52] utilize Transformer [32] for anomaly detection, leaning on the superiority modeling capacity of the self-attention mechanism for long-range relations.

However, all these works are based on the assumption that a sufficient training dataset with all normal instances can be acquired, which is very hard for real-world applications since each instance should be manually checked carefully. Additionally, once there exist abnormal instances in the training data, the performance of these OCC-based detection methods could be severely degraded [20], [21]. Therefore, instead of fitting the distribution of normal training datasets, Dai and Chen propose GANF [29] to detect MTS anomalies in an unsupervised manner. Inspired by them, we propose MTGFlow to facilitate the learning capacity and improve detection performance.

B. Graph Structure Learning for Anomaly Detection

Given a graph structure, graph neural networks [53], [54] have achieved great success in modeling intrinsic structure patterns. Wang et al. [55] extend the OCSVM to graph anomaly detection, combining the powerful representation ability of GNN. Tang et al. [56] analyze the anomaly detection from the graph spectral

domain and find graph anomalies lead to a shift of spectral energy distributions. They generate band-pass filters to capture this energy shift and further perform anomaly detection. However, In real scenarios, the prior graph structure is hard to provide. Therefore, it is important to learn the underlying graph structure via the graph embedding itself [57]. GDN [9] learns a directed graph via node embedding vectors. According to the cosine similarity of embedding vectors, top-K candidates of each node are considered to have dependencies. They rely on either the label supervision or normal training dataset. In the field of unsupervised anomaly detection of MTS, some recent work attempts to explore this area. GANF [29] uses a DAG to model the relationships between multiple sensors and then learns the structure of the DAG through continuous optimization while applying a simplified constraint to facilitate backward propagation. Our work, namely MTGFlow and MTGFlow_cluster, models the mutual complex dependencies as a fully connected dynamic graph via a self-attention mechanism, so that a much more flexible relation among entities can be represented.

C. Normalizing Flow for Anomaly Detection

Normalizing flow is a powerful technique for density estimation, and it has proven successful in image generation applications [30], [31]. Recently, it has also been applied to anomaly detection tasks under the assumption that anomalies occur in low-density regions, as seen in the works of DifferNet [58] and CFLOW-AD [59]. These approaches utilize normalizing flow to estimate the likelihoods of normal embedding and identify image defects by detecting embeddings that lie far from the dense region. GANF is a great approach to employ normalizing flow for unsupervised anomaly detection in MTS data. We build on this research direction by incorporating an entity/cluster-aware normalizing flow design to increase model capacity.

III. PRELIMINARY

We provide a brief introduction to normalizing flow in this section to better understand MTGFlow and its extension.

A. Self-Attention

Transformer [32] achieves unprecedented success for natural language processing (NLP), which largely benefits from self-attention superiority in modeling relations of input sequences. Give an input sequence, $X = (x_1, x_2, \dots, x_K)$, where $x_i \in \mathcal{R}^D$, and K and D are the sequence length and embedding dimension, respectively. Self-attention applies linear projections to get the query $Query \in \mathcal{R}^{K \times D}$ and key $Key \in \mathcal{R}^{K \times D}$ via learnable parameter matrices $W^{Query} \in \mathcal{R}^{D \times D}$ and $W^{Key} \in \mathcal{R}^{D \times D}$. After projections, attention scores are derived by the scaled dot product between $Query$ and Key , and a Softmax function is utilized to normalize the scores to (0,1).

$$Query = XW^{Query} \quad \text{and} \quad Key = XW^{Key},$$

$$Attention_matrix = Softmax\left(\frac{QueryKey^T}{\sqrt{D}}\right), \quad (1)$$

This design allows for flexible modeling of pairwise relationships within the neural network architecture. And, the resulting attention matrix serves as the quantification of these relationships.

B. Normalizing Flow

Normalizing flow is an unsupervised technique for estimating density, achieved by mapping the original distribution to an arbitrary target distribution using a sequence of invertible affine transformations. When density estimation on original data distribution $\mathcal{X}$ is intractable, an alternative option is to estimate z density on target distribution $\mathcal{Z}$. Specifically, suppose a source sample $x \in \mathcal{R}^D \sim \mathcal{X}$ and a target distribution sample $z \in \mathcal{R}^D \sim \mathcal{Z}$. Bijective invertible transformation $\mathcal{F}_\theta$ aims to achieve one-to-one mapping $z = f_\theta(x)$ from $\mathcal{X}$ to $\mathcal{Z}$. According to the change of variable formula, we can get

$$P_{\mathcal{X}}(x) = P_{\mathcal{Z}}(z) \left| \det \frac{\partial f_\theta}{\partial x^T} \right|. \quad (2)$$

Benefiting from the tractable jacobian determinants $|\det \frac{\partial f_\theta}{\partial x^T}|$. The objective of flow models is to achieve $\hat{z} = z$, where $\hat{z} = f_\theta(x)$. Parameters θ of f_θ can be directly estimated by MLE as follows

$$\begin{aligned} \theta^* &= \arg \max_{\theta} (\log(P_{\mathcal{X}}(x))) \\ &= \arg \max_{\theta} \left(\log \left(P_{\mathcal{Z}}(f_\theta(x)) \right) + \log \left(\left| \det \frac{\partial f_\theta}{\partial x^T} \right| \right) \right). \end{aligned} \quad (3)$$

Design of f_θ is an important topic in flow models. The core is to improve the flow model transferability under the premise of the tractable jacobian determinants. Representative flow model like RealNVP [30] utilizes the affine coupling, while MAF [31] applies autoregressive functions.

Flow models are able to achieve more superior density estimation performance when additional conditions C are input [60]. This is because C can usually provide relevant priors to accurately characterize the source distribution, such as time and position encoding [32]. Such the flow model is called conditional normalizing flow, and its corresponding mapping is derived as $z = f_\theta(x|C)$. Therefore, the training objective is rewritten as

$$\theta^* = \arg \max_{\theta} \left(\log \left(P_{\mathcal{Z}}(f_\theta(x|C)) \right) + \log \left(\left| \det \frac{\partial f_\theta}{\partial x^T} \right| \right) \right). \quad (4)$$

IV. METHOD

A. Problem Statement

Considering a MTS dataset $\mathcal{D} = (x_1, x_2, \dots, x_K)$ and $x_k \in \mathcal{R}^L$, where K represents the total number of entities, and L denotes the total number of observations of each entity. To preserve temporal correlations of the original series, we use a sliding window with size M and stride size S to sample the normalized MTS. M and S can be adjusted to obtain the training sample x^c , where c is the sampling count. x^c is short for $x^{cS:cS+M}$. Due to the fact that anomalies exhibit unusual temporal patterns with their neighbors, introducing temporal correlation to the input will benefit detection algorithms. In this

context, following [29], we focus on detecting window-level anomalies in this paper. That is when there exists an abnormal point in the chosen window, the window label is marked as the anomaly. Note that the time series is normalized by z-score from different entities.

$$x_k = \frac{x_k - \text{mean}(x_k)}{\text{std}(x_k)}, \quad (5)$$

where $\text{mean}(x_k)$ and $\text{std}(x_k)$ represent the mean and standard deviation of the k -th entity along the time dimension, respectively.

B. Overview of MTGFlow and MTGFlow_cluster

The main concepts behind MTGFlow and MTGFlow_cluster are to dynamically model mutual dependencies so that fine-grained density estimation of the multivariate time series can be obtained. This precise estimation allows for better detection of low-density regions, even in datasets with high anomaly contamination during training. The overview of MTGFlow and MTGFlow_cluster can be seen in Fig. 2. Concretely, each entity's temporal variations are modeled using an RNN, while a graph structure learning module is used to model the dynamic interdependencies. The RNN's time encoding output is then combined with the graph convolution operation, using the corresponding learned graph structure. These outputs are called spatio-temporal conditions, as they contain both temporal and structural information. In MTGFlow, these conditions are then input into the entity-aware normalizing flow module to achieve precise fine-grained density estimation. And MTGFlow_cluster is designed to first cluster these entities and then estimate the density via cluster-aware normalizing flow. As we do not have the cluster prior, we utilize time series clustering methods like KShape [37] to perform group assignments. Finally, the deviations of $\hat{z}$ and z are measured by log likelihoods, and all modules of MTGFlow and MTGFlow_cluster are optimized together using MLE.

C. Self-Attention Based Graph Structure Learning

In order to capture the mutual dependencies that exist between entities in a multivariate time series, we employ self-attention to learn a dynamic graph structure. Each entity is treated as a graph node. Given the window sequence $x^c \in \mathcal{R}^{K \times M}$, we calculate the query and key vectors for each node i as $x_i^c W^{Query}$ and $x_i^c W^{Key}$, where $W^{Query} \in \mathcal{R}^{M \times M}$ and $W^{Key} \in \mathcal{R}^{M \times M}$ are the query and key weights, respectively. The pairwise relationship e_{ij}^c between node i and node j at the c -th sampling count is then obtained as follows:

$$e_{ij}^c = \frac{(x_i^c W^{Query})(x_j^c W^{Key})^T}{\sqrt{M}}. \quad (6)$$

The attention score a_{ij}^c is used to quantify the pairwise relation from node i to node j , calculated by:

$$a_{ij}^c = \frac{\exp(e_{ij}^c)}{\sum_{j=1}^K \exp(e_{ij}^c)}, \quad A^c = \begin{bmatrix} a_{11}^c & \cdots & a_{1K}^c \\ \vdots & \ddots & \vdots \\ a_{K1}^c & \cdots & a_{KK}^c \end{bmatrix} \quad (7)$$

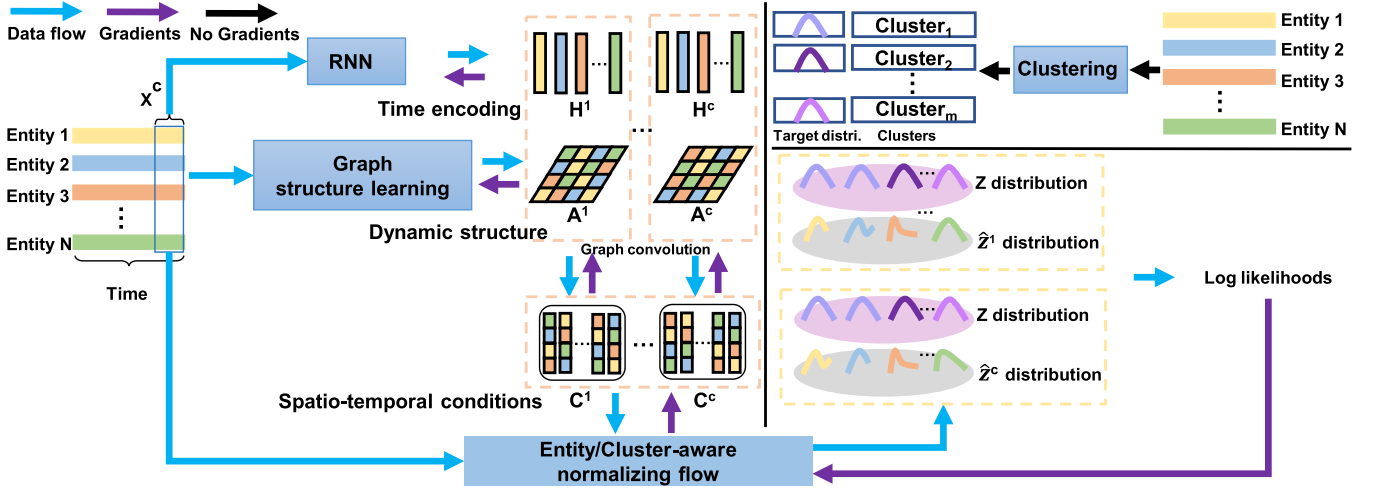


Fig. 2. Overview of the proposed MTGFlow and MTGFlow_cluster. Within a sliding window of size T , time series x^c is fed to the RNN module to capture the temporal correlations. Hidden states of RNN are regarded as time encoding, H^c . Meanwhile, x^c is also input to the graph structure learning module to capture dynamic interdependencies among entities, which are modeled as adjacency matrix A^c . The spatio-temporal conditions C^c are derived via the graph convolution operation for H^c and A^c . Finally, C^c is used to help entity/cluster-aware normalizing flow model to produce entity/cluster-specific density estimation for the distribution of time series.

The resulting attention score is used to construct the attention matrix A^c , which serves as the adjacency matrix of the learned graph. Since input time series are evolving over time, A^c also changes to capture the dynamic interdependencies.

D. Spatio-Temporal Condition

To accurately estimate the density of multiple time series, it is crucial to incorporate robust spatio-temporal contextual information. This involves modeling the underlying structure as a dynamic graph and considering both spatial and temporal correlations to effectively capture the features of the time series. We use recurrent neural networks (RNNs) to capture temporal correlations. For a window sequence of k -th entity, x_k^c , the time representation H_k^t at time $t \in [cS : cS + M)$ is derived by $H_k^t = \text{RNN}(x_k^t, H_k^{t-1})$, where RNN can be any sequence model such as LSTM [51] and GRU [61], and H_k^t is the hidden state of RNN. To obtain the spatio and temporal information C^t of all entities at t , a graph convolution operation is performed through the learned graph A^c . As mentioned in GANF, we also find that history information of the node itself helps enhance temporal relationships of time series. Hence, the spatio-temporal condition at t :

$$C^t = \text{ReLU}(A^c H^t W_1 + H^{t-1} W_2) W_3, \quad (8)$$

where W_1 and W_2 are graph convolution and history information weights, respectively. W_3 is used to improve the expression ability of condition representations. The spatio-temporal condition C^c for window c is the concatenation of C^t along the time axis.

E. Entity/Cluster-Aware Normalizing Flow

Distributions of individual entities have discrepancies because of their different work mechanisms, and thus their respective anomalies will generate distinct sparse characteristics. If we map time series from all entities to the same distribution $\mathcal{N}(0, I)$,

as does in GANF, then the description capacity of the model will be largely limited, and the unique inherent property of each entity will be ignored. Therefore, we design the entity-aware normalizing flow $z_k = f_\theta^k(x|C)$ to make more detailed density estimation, where x , C , k are the input sequence, condition and the k -th entity, respectively. Technically, in MTGFlow, for one entity, we assign the multivariate Gaussian distribution as the target distribution. As for MTGFlow_cluster, due to the absence of cluster priors, we leverage KShape for clustering all entities and subsequently assign the same target distribution to entities within a cluster. The covariance matrix of the above target distribution is the identity matrix I for better convergence. Moreover, in order to generate corresponding target distributions $\mathcal{Z}_k$, mean vectors $\mu_k \in R^M$ may be independently drawn from $\mathcal{N}(0, I)$ [62]. However, in our experiments, we find it better to keep each element of μ_k the same. Specifically, for the time series of the entity k , the entity-aware density estimation is given by:

$$\begin{aligned} P_{\mathcal{X}_k}(x_k) &= P_{\mathcal{Z}_k}(f_\theta^k(x_k|C)) \left| \det \frac{\partial f_\theta^k}{\partial x_k^T} \right|, \\ \mathcal{Z}_k &= \mathcal{N}(\mu_k, I), \\ \mu_k &\sim \mathcal{N}(0, 1), \end{aligned} \quad (9)$$

The cluster-aware density estimation is given by:

$$\begin{aligned} P_{\mathcal{X}_k}(x_k) &= P_{\mathcal{Z}_k}(f_\theta^k(x_k|C)) \left| \det \frac{\partial f_\theta^k}{\partial x_k^T} \right|, \\ \mathcal{Z}_k &= \mathcal{N}(\mu_m, I) \text{ if } k \in \text{cluster}_m, \\ \mu_m &\sim \mathcal{N}(0, 1), \\ \{\text{cluster}_1, \dots, \text{cluster}_m\} &= \text{KShape}\{\mathcal{D}\}, \end{aligned} \quad (10)$$

Note that when the cardinality of all clusters is one, MTGFlow is equal to MTGFlow_cluster. In such a case, model parameters

will increase with the number of entities/clusters. To mitigate this problem, we share entity/cluster-aware normalizing flow parameters across all entities. So, the density estimation for k reads:

$$P_{\mathcal{X}_k}(x_k) = P_{\mathcal{Z}_k}(f_\theta(x_k|C)) \left| \det \frac{\partial f_\theta}{\partial x_k^T} \right|. \quad (11)$$

F. Joint Optimization

As described above, MTGFlow and MTGFlow_cluster combine graph structure learning and RNN to capture the spatio and temporal dependencies on multiple time series. Then, derived spatio-temporal conditions are utilized to contribute to entity/cluster-aware normalizing flow estimating the density of time series. To avoid getting stuck in the local optimum for each module, we jointly optimize all modules. The whole parameters W^* are estimated via MLE.

$$\begin{aligned} W^* &= \arg \max_W \log(P_{\mathcal{X}}(x)) \\ &\approx \arg \max_W \frac{1}{N} \sum_{c=1}^N \frac{1}{K} \sum_{k=1}^K \log(P_{\mathcal{X}_k}(x_k^c)) \\ &\approx \arg \max_W \frac{1}{NK} \sum_{c=1}^N \sum_{k=1}^K \log \left(P_{\mathcal{Z}_k}(f_\theta(x_k^c|C_k^c)) \left| \det \frac{\partial f_\theta}{\partial x_k^c T} \right| \right) \\ &\approx \arg \max_W \frac{1}{NK} \sum_{c=1}^N \sum_{k=1}^K \left(-\frac{1}{2} \|\hat{z}_k^c - \mu_k\|_2^2 + \log \left| \det \frac{\partial f_\theta}{\partial x_k^c T} \right| \right. \\ &\quad \left. + Const \right), \end{aligned}$$

where N is the total number of windows, and $Const$ is equal to $-\frac{T}{2} * \log(2\pi)$.

G. Anomaly Detection and Interpretation

Based on the hypothesis that anomalies tend to be sparse on data distributions, low log likelihoods indicate that the observations are more likely to be anomalous.

1) *Anomaly Detection*: Taking the window sequence x^c as the input, the density of all entities can be estimated. The mean of the negative log likelihoods of all entities serves as the anomaly score S_c , which is calculated by:

$$S_c = -\frac{1}{K} \sum_{k=1}^K \log(P_{\mathcal{X}_k}(x_k^c)). \quad (12)$$

A higher anomaly score represents that x_k^c is located in the lower density region, indicating a higher possibility to be abnormal. Since abnormal series exist in the training set and validation set, we cannot directly set the threshold to label the anomaly, such as the maximum deviation in validation data [9]. Therefore, to reduce the anomaly disturbance, we store S_c of the whole training set, and the interquartile range (IQR) is used to set the threshold:

$$Threshold = Q_3 + 1.5 * (Q_3 - Q_1), \quad (13)$$

where Q_1 and Q_3 are 25th and 75th percentile of S_c .

2) *Anomaly Interpretation*: Abnormal behaviors of any entity could lead to the overall abnormal behavior of the whole window sequence. Naturally, we can get the anomaly score S_{ck} for entity k according to (12).

$$S_c = -\frac{1}{K} \sum_{k=1}^K \log(P_{\mathcal{X}_k}(x_k^c)) = \sum_{k=1}^K S_{ck}. \quad (14)$$

Since we map the time series of each entity into unique target distributions, different ranges of S_{ck} are observed. This bias will assign each entity to different weights in terms of its contribution to S_c . To circumvent the above-unexpected bias, we design the entity-specific threshold for each entity. Considering different scales of S_{ck} , IQR is used to set respective thresholds. Therefore, the threshold for S_{ck} is given as:

$$Threshold_k = \lambda_k(Q_3^k + 1.5 * (Q_3^k - Q_1^k)), \quad (15)$$

where Q_1^k and Q_3^k are 25th and 75th percentile of S_{ck} across all observations, respectively. And λ_k is used to adjust $Threshold_k$ because normal observations in different entities also fluctuate with different scales.

V. EXPERIMENT

A. Experiment Setup

1) *Dataset*: The commonly used public datasets for MTS anomaly detection in OCC are as follows:

- SWaT [63] is a collection of 51 sensor data from a real-world industrial water treatment plant, at the frequency of one second. The dataset provides ground truths of 41 attacks launched during 4 days.
- WADI [64] is a collection of 123 sensor and actuator data from WADI testbed, at the frequency of one second. The dataset provides ground truths of 15 attacks launched over 2 days.
- PSM [65] is a collection of multiple application server nodes at eBay with 25 features. The datasets provide ground truths created by experts over 8 weeks.
- MSL [49] is a collection of the sensor and actuator data of the Mars rover with 55 dimensions.
- SMD [7] is comprised of 28 small datasets from 28 machines at an internet company that records server metric data like CPU utilization, at the frequency of one minute. The dataset provides labeled anomalies for 5 weeks.

Since only normal time series are provided in these datasets for training in OCC setting, we follow the dataset setting of GANF [29] and split the original testing dataset by 60% for training, 20% for validation, and 20% for testing in SWaT. For other datasets, the training split contains 60% data, and the test split contains 40% data. Dataset statistics are provided in Table I.

In addition, we also consider the common OCC setting for a comprehensive evaluation. Under the OCC setting, the training datasets are required to be normal data, so we use the original training datasets of the above-mentioned datasets as training datasets. Besides, Wu et al. [44] provides elaborate and challenging datasets (UCR) including 250-sub univariate datasets

TABLE I
STATIC AND SETTINGS OF FIVE PUBLIC DATASETS

Dataset	Entity / metric number	Training set	Training set anomaly ratio (%)	Testing set	Testing set anomaly ratio (%)
SWaT	51	269951	17.7	89984	5.2
WADI	123	103680	6.4	69121	4.6
PSM	25	52704	23.1	35137	34.6
MSL	55	44237	14.7	29492	4.3
SMD	38	425052	4.2	283368	4.1

from diverse real scenarios (e.g., from biology and industry). For every sub-dataset, the training data is free of anomalies and the test data has only one short-period anomaly. Therefore, we follow the origin split for evaluation on UCR. Although MTGFlow is designed to detect multivariate time series anomaly detection, it can also be applied to univariate series. Note that MTGFlow is equivalent to MTGFlow_cluster when detecting univariate time series.

2) *Implementation Details*: For datasets of MTS, we set the window size as 60 and the stride size as 10. As for univariate dataset UCR, the duration of the anomalies is very short, and in some cases only a dozen sampling points. Hence, the window size is set as 10 for UCR. Adam optimizer with a learning rate of 0.002 is utilized to update all parameters. One layer of LSTM is employed to extract time representations in our experiment. One self-attention layer with 0.2 dropout ratio is adopted to learn the graph structure. We use MAF as the normalizing flow model. In MTGFlow_cluster, KShape is used to perform the whole time series clustering. The preset number of clusters is 20. For SWaT, one flow block and 512 batch size are employed. For other datasets, we arrange two flow blocks for it and set the batch size as 256. λ is set as 0.8 for thresholds of all entities. The epoch is 40 for all experiments, which are performed in PyTorch-1.7.1 with a single NVIDIA RTX 3090 24 GB GPU.¹

3) *Evaluation Metric*: As in previous works, MTGFlow and MTGFlow_cluster aim to detect the window-level anomalies, and labels are annotated as abnormal when there exists any anomalous time point. The performance is evaluated by the Area Under the Receiver Operating Characteristic curve (AUROC).

4) *Baselines*: To illustrate the effectiveness of our proposed method, we compare the SOTA semi-supervised and unsupervised density estimation methods.

- DeepSAD [39]. A semi-supervised method maps normal data into a preset hypersphere, and additional anomaly labels are leveraged to improve the detection performance.
- DeepSVDD [40]. An OCC-based method maps normal data into a preset hypersphere, and anomaly data are assumed to be located outside the hypersphere.
- ALOCC [41]. An OCC-based method reconstructs inputs via GAN and declares anomalies according to the output of the discriminator.
- DROCC [42]. An OCC-based method constructs robust representations for normal data. Anomalies will deviate from the learned representations.

- USAD [43]. An OCC-based method leverages Autoencoder to reconstruct inputs. Anomaly data suffer poor reconstruction quality.
- DAGMM [15]. A density estimation approach combines Autoencoder and Gaussian Mixture Model. Samples with high energy are declared as anomalies.
- GANF [29]. The density estimation approach joints DAG and flow model, and data on low-density regions are identified as anomalies.

5) *Performance*:

a) *Detection for multivariate time series*: We list the AUROC metric results in Table II, which is the average of five runs, in addition to the standard variance. Note that we do not provide the AUROC curves of SMD because it consists of 28 small datasets, where we test the performance on each of them and average all the results. MTGFlow and MTGFlow_cluster have superior performance over all the other seven baselines under the unsupervised setting. To present an intuitive result, AUROC curves are also presented in Fig. 3. Compared with MTGFlow and MTGFlow_cluster, DeepSVDD and DROCC project all training samples into the hypersphere so they cannot learn the accurate decision boundary distinguishing normal from abnormal samples. Adversarial learning used by ALOCC and USAD and the semi-supervised learning strategy in DeepSAD leverages a more informative training procedure to mitigate the effect of high anomaly contamination. As for DAGMM, it is restricted to the distribution estimation ability of GMM for multiple entities. Although GANF obtains a better result, its detection performance is still limited by inadequate dependence modeling and indiscriminative density estimation. Due to a much more flexible modeling structure, MTGFlow outperforms the above baseline methods. Moreover, we study log likelihoods for anomalies ranging from 2016/1/2 11:07:00 to 11:37:00 in Fig. 4. It is clear that log likelihoods are high for the normal series but lower for labeled abnormal ones (highlighted in red). This variation of log likelihoods validates that MTGFlow and MTGFlow_cluster can detect anomalies according to low-density regions of modeled distribution. Meanwhile, to investigate the anomaly discrimination ability of MTGFlow and MTGFlow_cluster, we present the normalized S_c for GANF, MTGFlow, and MTGFlow_cluster in Fig. 5. As it is displayed, for normal series, anomaly scores of MTGFlow and MTGFlow_cluster are more closed at 0 than those of GANF, and the overlap areas of normal and abnormal scores are also smaller, reducing the false positive ratio. The larger score discrepancy corroborates MTGFlow and MTGFlow_cluster superior detection performance.

¹Code is available at github.com/zqhang/MTGFLOW.

TABLE II
ANOMALY DETECTION PERFORMANCE OF AUROC(%) ON FIVE PUBLIC DATASETS

Dataset		DeepSVDD	ALOCC	DROCC	DeepSAD	USAD	DAGMM	GANF	MTGFlow	MTGFlow_cluster
Unsupervised setting	SWaT	66.8±2.0	77.1±2.3	72.6±3.8	75.4±2.4	78.8±1.0	72.8 ±3.0	79.8±0.7	84.8±1.5	83.1±1.3
	WADI	83.5±1.6	83.3±1.8	75.6±1.6	85.4±2.7	86.1±0.9	77.2±0.9	90.3±1.0	91.9±1.1	91.8±0.4
	PSM	67.5±1.4	71.8±1.3	74.3±2.0	73.2±3.3	78.0±0.2	64.6 ±2.6	81.8±1.5	85.7±1.5	87.1±2.4
	MSL	60.8±0.4	60.3±0.9	53.4±1.6	61.6±0.6	57.0±0.1	56.5 ±2.6	64.5±1.9	67.2±1.7	68.2±2.6
	SMD	75.5±15.5	80.5±11.1	76.7±8.7	85.9 ±11.1	86.9±11.7	78.0±9.2	89.2±7.8	91.3±7.6	-
OCC setting	SWaT	85.9±2.6	78.2±2.3	83.5±1.0	83.7±2.0	77.1±0.8	73.4±0.2	80.7±0.8	83.9±0.6	83.5±1.4
	WADI	85.5±0.4	88.7±0.9	89.0±1.1	88.2±1.5	88.5±1.2	84.8±2.3	91.7±0.9	92.2±1.0	92.1±0.8
	PSM	85.5±1.9	74.7±1.7	83.4±1.1	84.9±1.0	80.0±0.8	69.4±1.6	85.2±0.9	86.1±1.6	87.9±2.0
	MSL	61.9±0.8	61.8 ±0.9	56.8±0.6	62.8±0.7	58.9±0.6	59.3±0.6	66.7±1.4	68.3±1.3	68.9±1.6
	SMD	86.4±9.8	81.9±9.4	79.1±9.7	89.4±8.7	87.8±11.3	79.1±9.2	90.2±8.4	91.8±6.4	-

We test MTGFlow and MTGFlow_Cluster in unsupervised and OCC settings.

The bold values indicates the best result.

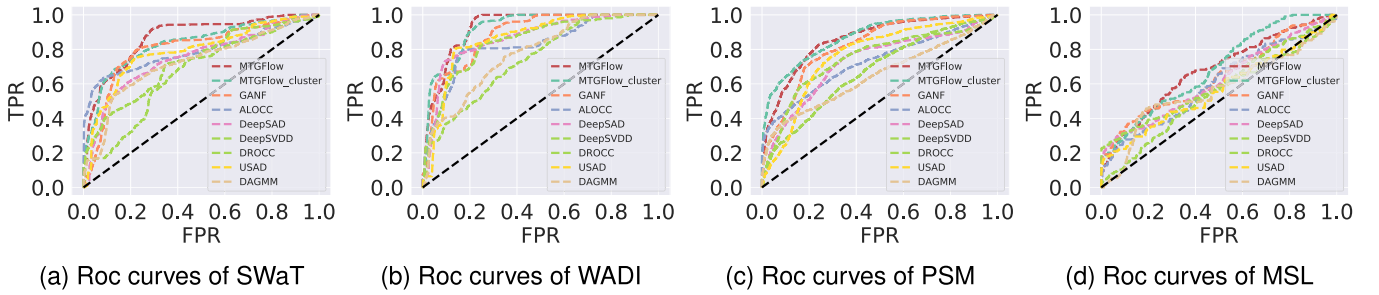


Fig. 3. Comparison with advanced methods on ROC curves for four datasets.

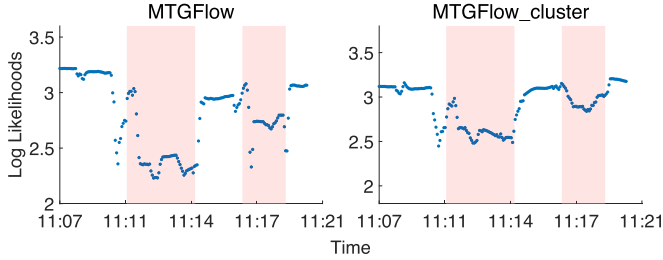


Fig. 4. Log likelihoods for anomalies on MTGFlow and MTGFlow_cluster.

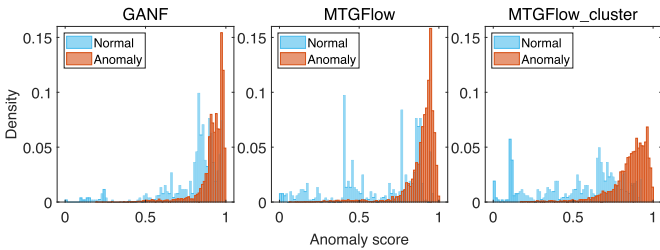


Fig. 5. Discrimination comparison on SWaT between GANF, MTGFlow, and MTGFlow_cluster.

For the OCC setting, MTGFlow and MTGFlow_cluster also achieve promising performance. Interestingly, the performance of MTGFlow and MTGFlow_cluster are close to that in the unsupervised setting, while other baselines have a tremendous performance improvement on different datasets. Specifically, DeepSVDD has a significant gain from 66.8 AUROC% to 85.9

AUROC% on SWaT and from 67.5 AUROC% to 85.5 AUROC% on PSM. Besides, DROCC also obtains the advancement from 75.6 AUROC% to 89.0 AUROC% on WADI. These observations reflect that the anomaly contamination severely degrades the detection performance of the above OCC-based methods. Therefore, such contrast illustrates that our method is more robust to anomaly contamination and thus applicable to both two scenarios.

Although MTGFlow and MTGFlow_cluster are superior to the baselines, we observe that MTGFlow_cluster does not consistently outperform MTGFlow according to the results. We attribute this performance degradation (in the case of SWaT and WADI datasets) to the inappropriate cluster assignments. Because of the lack of prior clusters, we use KShape to provide the pseudo cluster label. Although KShape is a highly accurate and scalable time series clustering algorithm, the entity allocation is sensitive to the number of predefined clusters. If two or more entities with distinct characteristics are clustered into a group, MTGFlow_cluster will combine multiple underlying data manifolds into a single distribution, thereby hindering the accurate estimation for each entity. In our experiments, we apply default parameters to all datasets, i.e., the number of predefined clusters is set to 20 across different datasets such as SWaT and WADI. This cluster number may be not optimal for all datasets. To investigate the effect of cluster number, we perform a detailed analysis in Section V-B2.

b) Detection for univariate time series: As for univariate series, we evaluate our method on UCR with other baselines. Since UCR has 250 sub-datasets, we first average the results

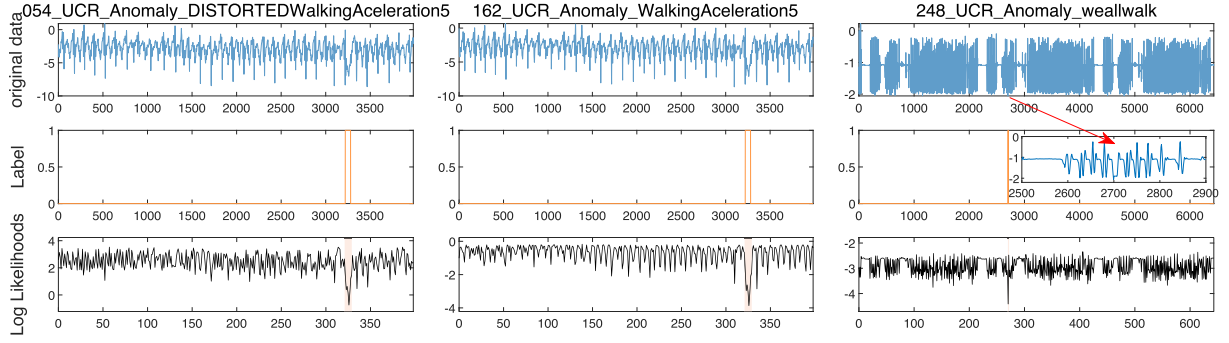


Fig. 6. Log likelihoods for anomalies on UCR. *Top*: The original test data in respective sub-datasets are presented. *Middle*: The corresponding labels provided in these sub-datasets are provided. Normal time points are labeled as 0, while abnormal ones are labeled as 1. *Bottom*: Log likelihoods are obtained from our method, and the ground truths of abnormal points are highlighted in red.

TABLE III
PERFORMANCE OF AUROC(%) ON UCR

Dataset	DeepSVDD	ALOCC	DROCC	DeepSAD
UCR	52.1±1.3	58.6±1.9	59.3±2.4	60.5±2.1
	USAD	DAGMM	GANF	MTGFlow
	56.7±0.0	62.5±0.5	63.4±0.2	67.4±0.4

The bold values indicates the best result.

TABLE IV
MODULE ABLATION STUDY (AUROC%)

	Graph	Entity-aware	SWaT	WADI
MTGFlow/(G, E)	✗	✗	78.3±0.9	89.7±0.5
MTGFlow/G	✗	✓	82.4±1.0	91.3±0.4
MTGFlow/E	✓	✗	81.2±1.1	91.0±0.7
MTGFlow	✓	✓	84.8±1.5	91.9±1.1

The bold values indicates the best result.

over these sub-datasets and then average the above results over five runs with a standard variance. From Table III, our method still obtains the best performance. As suggested in [44], We visual the original data of three sub-datasets (i.e., the unexpected acceleration of distorted walking and walking) and present the obtained log likelihoods from our method in Fig. 6. From the figure, anomalous points in 054_UCR and 162_UCR datasets are not sudden extreme values, which may be identified via an empirical threshold. Their values are in the normal range but exhibit an unusual pattern caused by the unexpected acceleration. Furthermore, in addition to the difficulty mentioned above, the duration of the anomaly in 248_UCR dataset is also very short (i.e., just five abnormal points). These challenges make it more difficult to detect such anomalies. Bypassing these problems, our method estimates the density of data points rather than focusing on their numerical values and successfully detects such anomalies according to the figure. Moreover, we can observe that anomalous points incur lower log likelihoods than normal points. And, the prominent peaks show that MTGFlow can discriminate anomalies clearly.

B. Ablation Study

To study the effectiveness of our proposed module, we perform several ablation studies, including module ablation, hyperparameter robustness, and anomaly ratio analysis. We mainly focus on the analysis of MTGFlow because it is the special case of MTGFlow_cluster, where the cardinality of each cluster is one. As for the effect of cluster quantities, we will analyze it in Section V-B2.

1) *Module Ablation Study*: To test the validity of each designed module, we give several ablation experiments. We denote MTGFlow without graph and entity-aware normalizing

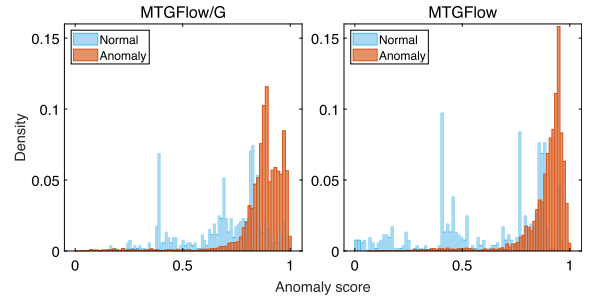


Fig. 7. Comparison on normalized anomaly scores between MTGFlow and MTGFlow/G.

flow as MTGFlow/(G, E), MTGFlow only without graph as MTGFlow/G, and MTGFlow only without entity-aware normalizing flow as MTGFlow/E. Results are presented in Table IV, where MTGFlow/(G, E) obtains the worst performance. It is attributed to the lack of relational modeling among entities and indistinguishable density estimation. Applying graph structure learning to model pairwise relations, MTGFlow/E achieves better performance. Also, considering more fine-grained density estimation, MTGFlow/G achieves an improvement over MTGFlow/(G, E). Integrating these two modules, MTGFlow accomplishes the best results. To provide a more intuitive result, we visualize the normalized anomaly score distributions of MTGFlow and MTGFlow/G in Fig. 7. Such a dynamic graph design can push anomaly scores of normal series to 0 and enlarge the margin between normal and abnormal series. Also, we present the normalized anomaly score distribution for normal and abnormal series in Fig. 8. We observe that the margin between normal and abnormal samples in MTGFlow is larger than that

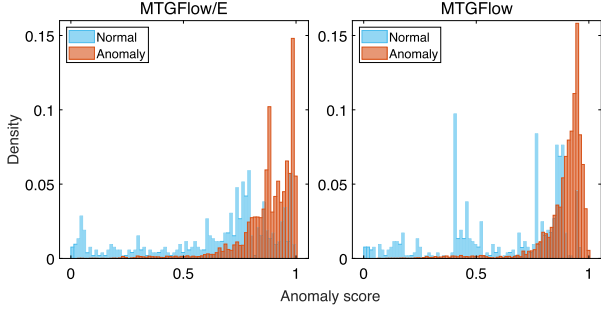


Fig. 8. Comparison on normalized anomaly scores between MTGFlow and MTGFlow/E.

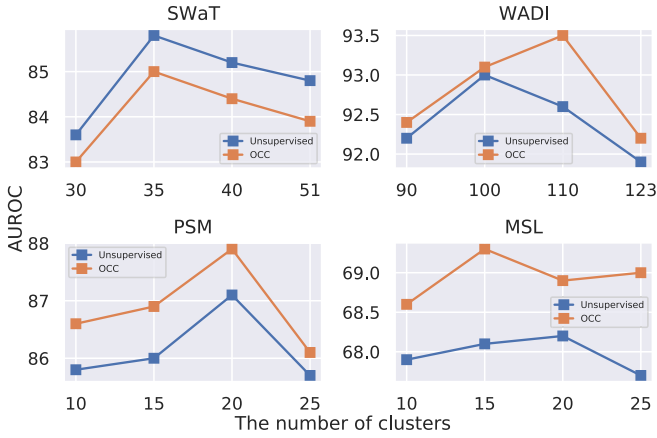


Fig. 9. Effect of the cluster number in MTGFlow_cluster.

in MTGFlow/E. This demonstrates that MTGFlow amplifies the discrepancy between normal and abnormal samples with the help of entity-specific density estimation.

2) *Clustering Quantity Study*: The pre-set cluster number is important to the performance of MTGFlow_cluster. Hence, we investigate the effect of the number of clusters on detection performance under unsupervised and OCC settings. Since the number of entities varies from different datasets, we assign different cluster numbers for them, as shown in Fig. 9. Take SWaT for example, the detection performance reaches the peak when the cluster number is 35. This illustrates that the appropriate number of clusters can facilitate the detection performance for both settings. If the number of clusters is unexpectedly small, some entities with completely different characteristics are gathered into the same cluster and transformed into the same target distribution. In this case, the detection performance will decrease, as presented by the cluster number from 35 to 30 in SWaT. Conversely, when the number of clusters is excessive, too many unnecessary target distributions need to be estimated, which may also result in performance degradation, as shown by the cluster number from 35 to 51 in SWaT. Similar phenomena are also observed across other datasets. We refer to MTGFlow_cluster as MTGFlow_cluster* when the appropriate cluster number is assigned. As shown in Table VI, MTGFlow_cluster* achieves better detection performance than MTGFlow.

3) *Hyperparameter Robustness*: We conduct a comprehensive study on the choice of hyperparameters, the results are

TABLE V
ABLATION STUDY OF HYPERPARAMETERS (AUROC%)

Blocks		1	2	3
Window size				
SWaT	40	81.4±3.2	82.7±2.1	81.7±0.9
	60	84.8±1.5	83.6±2.0	83.1±0.9
	80	82.8±1.0	82.7±0.8	83.4±0.6
	100	82.6±0.5	83.4±0.9	83.5±0.6
	120	83.2±2.0	83.4±2.3	84.5±2.6
WADI	40	90.8±1.3	91.7±1.2	91.7±1.3
	60	89.2±1.9	91.9±1.1	91.5±0.8
	80	89.8±2.0	90.7±0.8	91.7±0.7
	100	89.6±1.1	90.9±0.8	91.8±0.6
	120	88.6±1.4	91.0±0.6	91.5±0.9

TABLE VI
PERFORMANCE COMPARISON ON MTGFlow AND MTGFlow_CLUSTER*

Methods		SWaT	WADI	PSM	MSL
Unsupervised setting	MTGFlow_cluster*	85.8	93.0	87.1	68.2
	MTGFlow	84.8	91.9	85.7	67.2
OCC setting	MTGFlow_cluster*	85.0	93.5	87.9	69.3
	MTGFlow	83.9	92.2	86.1	68.3

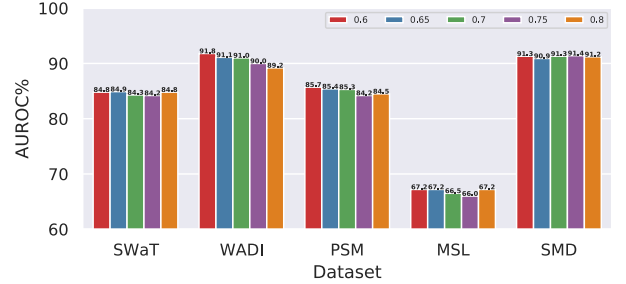


Fig. 10. Effect of anomaly contamination ratio.

shown in Table V. Concretely, we conduct experiments with various sizes for the sliding window and the number of the normalizing flow blocks in Table V. When the window size is small, such as 40, 60, and 80, the increase in the number of blocks does not necessarily improve anomaly detection performance. A larger model may cause overfitting to the whole distribution, where abnormal sequences are undesirably located in high-density regions of this distribution. When the window size is large (i.e., 80, 100, and 120), the distribution to be estimated is more high-dimensional so that model needs more capacity. Therefore, detection performance derives the average gain with blocks increasing due to more accurate distribution modeling.

4) *Anomaly Ratio Analysis*: To further investigate the influence of anomaly contamination rates, we vary training splits to adjust anomalous contamination rates. For all the above-mentioned datasets, the training split increases from 60% to 80% with a 5% stride. We present an average result over five runs in Fig. 10. Taking SWaT as an example, the corresponding anomaly ratios of training datasets on SWaT are 17.7%, 16.7%, 15.7%, 14.6%, and 13.9%, respectively. The detection performance of

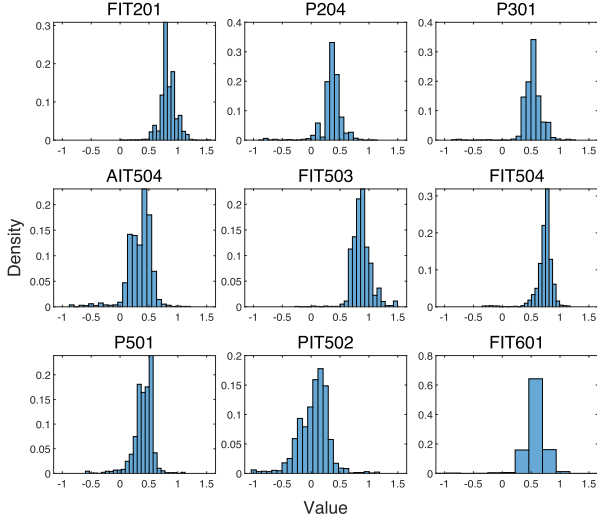


Fig. 11. Transformed distributions of multiple entities.

MTGFlow fluctuates in a small range, which shows MTGFlow is robust to the degree of anomaly contamination ratio. The same phenomenon can be seen in the rest datasets. Consequently, we can conclude that the anomaly detection performance of MTGFlow remains at a stable high level as the anomaly contamination ratio of the training dataset rises.

C. Result Analysis

In order to further investigate the effectiveness of MTGFlow, we give a detailed analysis based on SWaT dataset.

1) *Entity-specific Density Estimation*: We further explore whether the distributions of all entities are transformed into different target distributions to verify our entity-aware design. Since the window size is 60, the corresponding transformed distributions are also 60-dimensional distributions. Every single dimension of the multivariate Gaussian distribution is a Gaussian distribution. For better visualization, we present the 0th dimension of the transformed distributions across the whole test dataset in Fig. 11. Nine distributions of different entities are displayed. It can be seen that these distributions have been projected as unique distributions. Moreover, these distributions are successfully converted to preset Gaussian distributions with different mean vectors. The one-to-one mapping models entity-specific distributions and captures their respective sparse characteristics of anomalies, thus mitigating the above-mentioned performance sacrifice.

2) *Dynamic Graph Structure*: Interdependencies among entities are not guaranteed to be immutable. In fact, pairwise relations evolve with time. Benefiting from self-attention, MTGFlow can model this characteristic into a dynamic graph structure. We treat the attention matrix as the graph adjacent matrix. An empirical threshold of 0.15 is set for the adjacency matrix to show an intuitive learned graph structure in the test split. In Fig. 12, the node size represents its node degrees, the arrow direction represents the learned directed dependence and the arrow width indicates the weight of the corresponding interdependencies. The graph structure at 2016/1/1 14:00:00 is centered on the sensor *P201*, while the edges in the graph

have completely changed and the center has shifted from *P201* to *AIT503* at 2016/1/2 7:00:00. This alteration of the graph structure may result from changes in working condition of the water treatment plant. Besides, two similar graph structures can be found at 2016/1/2 13:00:00 and 2016/1/2 14:00:00. This suggests that the graph structure will be consistent if the interdependencies remain unchanged over a period of time, possibly due to repetitive work patterns of entities. In addition, the main pairwise relations (thick arrow) at 2016/1/1 14:00:00 are similar to the ones at 2016/1/2 14:00:00, both centered on *P201*. It indicates that the interdependencies on multiple sensors are periodic. We also find mutual interdependencies from learned graph structures, such as the edges between *P201* and *AIT201* at 2016/1/2 13:00:00. We summarize the findings:

- 1) Dynamic interdependencies among multiple entities.
- 2) Consistent interdependencies among multiple entities.
- 3) Periodic interdependencies among multiple entities.
- 4) Mutual interdependencies among multiple entities.

Therefore, it is necessary to use a dynamic graph to model such changeable interdependencies.

3) *Distinct Sparse Characteristics*: To demonstrate that the sparse characteristics vary with different entities, we study changes of S_{ck} along time on SWaT. As shown in Fig. 13, S_{ck} of *AIT201*, *P102*, *FIT502*, *AIT502*, *P403*, and *LIT301* are presented. The highlighted regions denote marked anomalies. For a better illustration, we divide the anomalous regions as A_1 , A_2 , A_3 , A_4 , and A_5 along the timeline. We can observe that different entities react to different anomalies because of their different work mechanisms. Specifically, *AIT201* and *AIT502* has obvious fluctuations at A_4 , while *P102* reacts to A_2 . In addition, *FIT502* is sensitive to A_4 and A_5 , yet *P403* has responses in the face of A_2 and A_4 . But *LIT301* is sensitive to A_2 , A_3 , A_4 , and A_5 . Nevertheless, MTGFlow is still able to accurately distinguish and detect these anomalies.

D. Experimental Summary

Without the requirement for any labeled data, MTGFlow and MTGFlow_cluster have demonstrated their capabilities to identify anomalies by leveraging their intrinsic low data density in Section V. We attribute the superior performance of MTGFlow and MTGFlow_cluster to the following parts:

- Individual anomalous time series from different entities present diverse sparse characteristics. Instead of mapping all these individual time series to the same latent space, we introduce an entity-aware normalizing flow to model entity-specific density estimation. This allows each entity to be associated with a unique target distribution, which results in an improvement in performance by 4.1% AUROC on SWaT and 1.6% AUROC on WADI.
- Since interdependencies among entities are not immutable, they actually evolve over time. Rather than modeling such interdependencies as static, we propose a dynamic graph structure to capture these changeable interrelationships. This innovation achieves the performance improvement of 2.4% on SWaT and 0.6% AUROC on WADI.

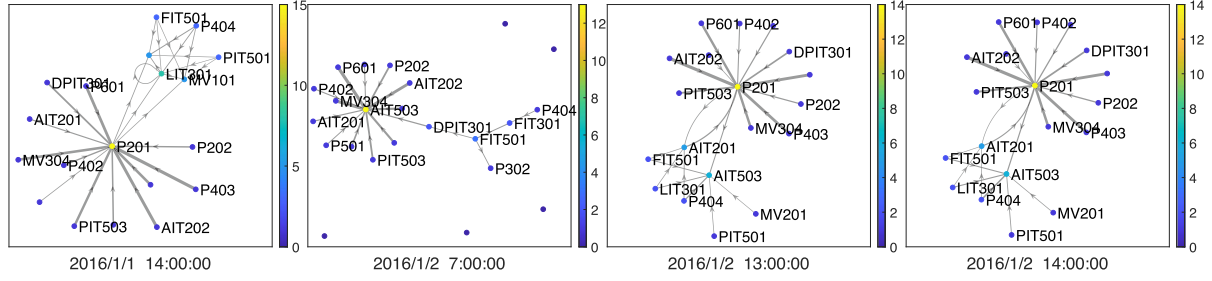
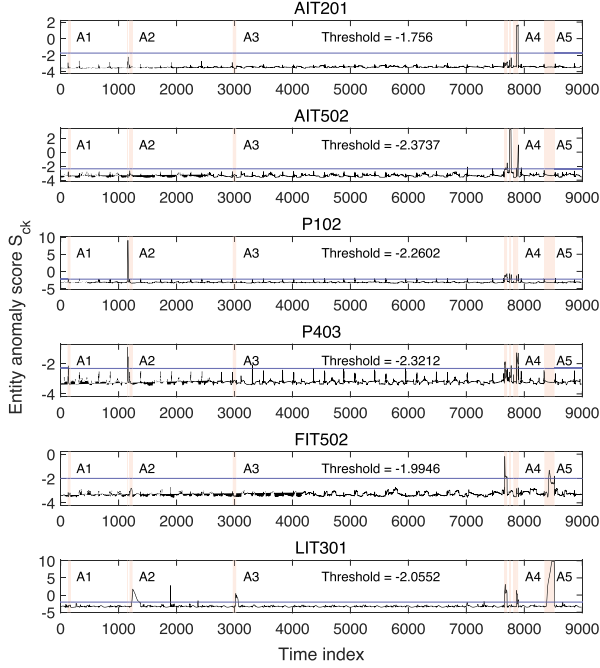


Fig. 12. Dynamic graph structure in MTGFlow.

Fig. 13. Variation of log likelihoods for different entities on the whole testing dataset (Anomalies are highlighted in red, and the blue line is the threshold according to S_{ck}).

- In reality, the characteristics of entities are strongly connected to their positions in common scenarios. In contrast to the approach of mapping all entities to unique distributions like MTGFlow, MTGFlow_cluster further proposes the cluster-aware normalizing flow to obtain the commonality of these characteristics while maintaining inter-cluster uniqueness. This cluster strategy empowers MTGFlow_cluster to outperform MTGFlow by 1.0% AUROC on SWaT and 1.1% AUROC on WADI.

VI. CONCLUSION

In this work, we proposed MTGFlow and MTGFlow_cluster, unsupervised anomaly detection approaches for MTS based on the dataset with absolute zero known label. They first model the distribution of training data and then estimate the density of test samples, which are identified anomalies when they lie in low-density regions of this distribution. Extensive experiments on real-world datasets demonstrate their superiority, even if there exists high anomaly contamination. MTGFlow and MTGFlow_cluster achieve SOTA anomaly detection methods

for MTS and univariate series in both unsupervised and OCC settings. The superior anomaly detection performance of MTGFlow and MTGFlow_cluster is attributed to dynamic graph structure learning and entity/cluster-aware density estimation. In addition, we explore various interdependencies that exist between individual entities from the learned dynamic graph structure. And a detected anomaly can be understood and localized via entity anomaly scores. In the future, we plan to apply our model to more flow models and further improve the practicality of our approach.

REFERENCES

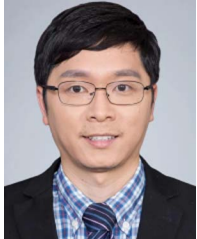
- [1] S. He, K. Shi, C. Liu, B. Guo, J. Chen, and Z. Shi, "Collaborative sensing in Internet of Things: A comprehensive survey," *IEEE Commun. Surveys Tuts.*, vol. 24, no. 3, pp. 1435–1474, Third Quarter, 2022.
- [2] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection for discrete sequences: A survey," *IEEE Trans. Knowl. Data Eng.*, vol. 24, no. 5, pp. 823–839, May 2012.
- [3] S. R. Gaddam, V. V. Phoha, and K. S. Balagani, "K-means+ID3: A novel method for supervised anomaly detection by cascading K-means clustering and ID3 decision tree learning methods," *IEEE Trans. Knowl. Data Eng.*, vol. 19, no. 3, pp. 345–354, Mar. 2007.
- [4] Z. Yang, L. He, H. Yu, C. Zhao, P. Cheng, and J. Chen, "Detecting PLC intrusions using control invariants," *IEEE Internet Things J.*, vol. 9, no. 12, pp. 9934–9947, Jun. 2022.
- [5] H. Pu, L. He, P. Cheng, M. Sun, and J. Chen, "Security of industrial robots: Vulnerabilities, attacks, and mitigations," *IEEE Netw.*, vol. 37, no. 1, pp. 111–117, Jan./Feb. 2023.
- [6] B. Schölkopf et al., "Support vector method for novelty detection," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 1999, pp. 582–588.
- [7] Y. Su, Y. Zhao, C. Niu, R. Liu, W. Sun, and D. Pei, "Robust anomaly detection for multivariate time series through stochastic recurrent neural network," in *Proc. 25th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2019, pp. 2828–2837.
- [8] X. Chen et al., "DAEMON: Unsupervised anomaly detection and interpretation for multivariate time series," in *Proc. IEEE 37th Int. Conf. Data Eng.*, 2021, pp. 2225–2230.
- [9] A. Deng and B. Hooi, "Graph neural network-based anomaly detection in multivariate time series," in *Proc. AAAI Conf. Artif. Intell.*, 2021, pp. 4027–4035.
- [10] J. Xu, H. Wu, J. Wang, and M. Long, "Anomaly transformer: Time series anomaly detection with association discrepancy," 2021, *arXiv:2110.02642*.
- [11] C. Zhang et al., "A deep neural network for unsupervised anomaly detection and diagnosis in multivariate time series data," in *Proc. AAAI Conf. Artif. Intell.*, 2019, pp. 1409–1416.
- [12] L. Ruff et al., "A unifying review of deep and shallow anomaly detection," in *Proc. IEEE*, vol. 109, no. 5, pp. 756–795, May 2021.
- [13] A. Goode, B. Hooi, S. K. Ng, and W. S. Ng, "LUNAR: Unifying local outlier detection methods via graph neural networks," 2021, *arXiv:2112.05355*.
- [14] L. Zhang, J. Zhao, and W. Li, "Online and unsupervised anomaly detection for streaming data using an array of sliding windows and PDDs," *IEEE Trans. Cybern.*, vol. 51, no. 4, pp. 2284–2289, Apr. 2021.

- [15] B. Zong et al., "Deep autoencoding gaussian mixture model for unsupervised anomaly detection," in *Proc. Int. Conf. Learn. Representations*, 2018, pp. 1–19.
- [16] C. Qiu, A. Li, M. Kloft, M. Rudolph, and S. Mandt, "Latent outlier exposure for anomaly detection with contaminated data," 2022, *arXiv:2202.08088*.
- [17] S. Liu et al., "Time series anomaly detection with adversarial reconstruction networks," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 4, pp. 4293–4306, Apr. 2023.
- [18] K. Zhang, C. Li, and Q. Yang, "TriD-MAE: A generic pre-trained model for multivariate time series with missing values," in *Proc. 32nd ACM Int. Conf. Inf. Knowl. Manage.*, 2023, pp. 3164–3173.
- [19] C. Zhang, S. Bengio, L. Hardt, B. Recht, and O. Vinyals, "Understanding deep learning (still) requires rethinking generalization," *Commun. ACM*, vol. 64, no. 3, pp. 107–115, 2021.
- [20] S. Wang et al., "Effective end-to-end unsupervised outlier detection via inlier priority of discriminative network," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2019, Art. no. 536.
- [21] N. Huyen, D. Quan, X. Zhang, X. Liang, J. Chanussot, and L. Jiao, "Unsupervised outlier detection using memory and contrastive learning," 2021, *arXiv:2107.12642*.
- [22] M. Gupta, J. Gao, C. C. Aggarwal, and J. Han, "Outlier detection for temporal data: A survey," *IEEE Trans. Knowl. Data Eng.*, vol. 26, no. 9, pp. 2250–2267, Sep. 2011.
- [23] G. Pang, L. Cao, and C. Aggarwal, "Deep learning for anomaly detection: Challenges, methods, and opportunities," in *Proc. 14th ACM Int. Conf. Web Search Data Mining*, New York, NY, USA, 2021, pp. 1127–1130. [Online]. Available: <https://doi.org/10.1145/3437963.3441659>
- [24] R. Wang et al., "Deep learning for anomaly detection," in *Proc. 26th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, New York, NY, USA, 2020, pp. 3569–3570. [Online]. Available: <https://doi.org/10.1145/3394486.3406481>
- [25] D. Salinas, V. Flunkert, J. Gasthaus, and T. Januschowski, "DeepAR: Probabilistic forecasting with autoregressive recurrent networks," *Int. J. Forecasting*, vol. 36, no. 3, pp. 1181–1191, 2020.
- [26] K. Rasul, C. Seward, I. Schuster, and R. Vollgraf, "Autoregressive denoising diffusion models for multivariate probabilistic time series forecasting," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 8857–8868.
- [27] S. Feng, K. Xu, J. Wu, P. Wu, F. Lin, and P. Zhao, "Multi-scale attention flow for probabilistic time series forecasting," 2022, *arXiv:2205.07493*.
- [28] K. Rasul, A.-S. Sheikh, I. Schuster, U. Bergmann, and R. Vollgraf, "Multivariate probabilistic time series forecasting via conditioned normalizing flows," 2020, *arXiv: 2002.06103*.
- [29] E. Dai and J. Chen, "Graph-augmented normalizing flows for anomaly detection of multiple time series," in *Proc. Int. Conf. Learn. Representations*, 2021, pp. 1–16.
- [30] L. Dinh, J. Sohl-Dickstein, and S. Bengio, "Density estimation using real NVP," 2016, *arXiv:1605.08803*.
- [31] G. Papamakarios, T. Pavlakou, and I. Murray, "Masked autoregressive flow for density estimation," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2017, pp. 2338–2347.
- [32] A. Vaswani et al., "Attention is all you need," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2017, pp. 5998–6008.
- [33] P. P. Rodrigues, J. Gama, and J. Pedroso, "Hierarchical clustering of time-series data streams," *IEEE Trans. Knowl. Data Eng.*, vol. 20, no. 5, pp. 615–627, May 2008.
- [34] Z. Li, Y. Zhao, R. Liu, and D. Pei, "Robust and rapid clustering of KPIs for large-scale anomaly detection," in *Proc. IEEE/ACM 26th Int. Symp. Qual. Serv.*, 2018, pp. 1–10.
- [35] G. Liu, J. Si, W. Meng, Q. Yang, and C. Li, "Wind turbine fault detection with multimodule feature extraction network and adaptive strategy," *IEEE Trans. Instrum. Meas.*, vol. 72, 2023, Art. no. 3504613.
- [36] A. Aboah, "A vision-based system for traffic anomaly detection using deep learning and decision trees," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 4207–4212.
- [37] J. Paparrizos and L. Gravano, "k-Shape: Efficient and accurate clustering of time series," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2015, pp. 1855–1870.
- [38] Q. Ma, J. Zheng, S. Li, and G. W. Cottrell, "Learning representations for time series clustering," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett, Eds., 2019, pp. 3776–3786. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2019/file/1359aa933b48b754a2f54adb688bfa77-Paper.pdf
- [39] L. Ruff et al., "Deep semi-supervised anomaly detection," 2019, *arXiv: 1906.02694*.
- [40] L. Ruff et al., "Deep one-class classification," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 4393–4402.
- [41] M. Sabokrou, M. Fathy, G. Zhao, and E. Adeli, "Deep end-to-end one-class classifier," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 32, no. 2, pp. 675–684, Feb. 2021.
- [42] S. Goyal, A. Raghunathan, M. Jain, H. V. Simhadri, and P. Jain, "DROCC: Deep robust one-class classification," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 3711–3721.
- [43] J. Audibert, P. Michiardi, F. Guyard, S. Marti, and M. A. Zuluaga, "USAD: Unsupervised anomaly detection on multivariate time series," in *Proc. 26th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2020, pp. 3395–3404.
- [44] R. Wu and E. J. Keogh, "Current time series anomaly detection benchmarks are flawed and are creating the illusion of progress," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 3, pp. 2421–2429, Mar. 2023.
- [45] Q. Zhou, J. Chen, H. Liu, S. He, and W. Meng, "Detecting multivariate time series anomalies with zero known label," in *Proc. AAAI Conf. Artif. Intell.*, 2023, pp. 4963–4971. [Online]. Available: <https://ojs.aaai.org/index.php/AAAI/article/view/25623>
- [46] R. Moghaddass and J. Wang, "A hierarchical framework for smart grid anomaly detection using large-scale smart meter data," *IEEE Trans. Smart Grid*, vol. 9, no. 6, pp. 5820–5830, Nov. 2018.
- [47] M. Ghafouri, M. Au, M. Kassouf, M. Debbabi, C. Assi, and J. Yan, "Detection and mitigation of cyber attacks on voltage stability monitoring of smart grids," *IEEE Trans. Smart Grid*, vol. 11, no. 6, pp. 5227–5238, Nov. 2020.
- [48] R. Chalapathy and S. Chawla, "Deep learning for anomaly detection: A survey," 2019, *arXiv: 1901.03407*.
- [49] K. Hundman, V. Constantinou, C. Laporte, I. Colwell, and T. Soderstrom, "Detecting spacecraft anomalies using LSTMs and nonparametric dynamic thresholding," in *Proc. 24th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2018, pp. 387–395.
- [50] P. Malhotra, A. Ramakrishnan, G. Anand, L. Vig, P. Agarwal, and G. Shroff, "LSTM-based encoder-decoder for multi-sensor anomaly detection," 2016, *arXiv:1607.00148*.
- [51] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [52] S. Tuli, G. Casale, and N. R. Jennings, "TranAD: Deep transformer networks for anomaly detection in multivariate time series data," 2022, *arXiv:2201.07284*.
- [53] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," 2016, *arXiv:1609.02907*.
- [54] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How powerful are graph neural networks?," 2018, *arXiv: 1810.00826*.
- [55] J. Tang, J. Li, Z. Gao, and J. Li, "Rethinking graph neural networks for anomaly detection," in *Proc. 39th Int. Conf. Mach. Learn.*, K. Chaudhuri, S. Jegelka, L. Song, C. Szepesvari, G. Niu, and S. Sabato, Eds., 2022, pp. 21 076–21 089. [Online]. Available: <https://proceedings.mlr.press/v162/tang22b.html>
- [56] X. Wang, B. Jin, Y. Du, P. Cui, Y. Tan, and Y. Yang, "One-class graph neural networks for anomaly detection in attributed networks," *Neural Comput. Appl.*, vol. 33, pp. 12 073–12 085, 2021.
- [57] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Lio, and Y. Bengio, "Graph attention networks," 2017, *arXiv: 1710.10903*.
- [58] M. Rudolph, B. Wandt, and B. Rosenhahn, "Same same but Different: Semi-supervised defect detection with normalizing flows," in *Proc. IEEE/CVF Winter Conf. Appl. Comput. Vis.*, 2021, pp. 1907–1916.
- [59] D. Gudovskiy, S. Ishizaka, and K. Kozuka, "CFLOW-AD: Real-time unsupervised anomaly detection with localization via conditional normalizing flows," in *Proc. IEEE/CVF Winter Conf. Appl. Comput. Vis.*, 2022, pp. 98–107.
- [60] L. Ardizzone, C. Lüth, J. Kruse, C. Rother, and U. Köthe, "Guided image generation with conditional invertible neural networks," 2019, *arXiv: 1907.02392*.
- [61] K. Cho et al., "Learning phrase representations using RNN encoder-decoder for statistical machine translation," 2014, *arXiv:1406.1078*.
- [62] P. Izmailov, P. Kirichenko, M. Finzi, and A. G. Wilson, "Semi-supervised learning with normalizing flows," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 4615–4630.
- [63] J. Goh, S. Adepu, K. N. Junejo, and A. Mathur, "A dataset to support research in the design of secure water treatment systems," in *Proc. Int. Conf. Crit. Inf. Infrastructures Secur.*, Springer, 2016, pp. 88–99.
- [64] C. M. Ahmed, V. R. Palleti, and A. P. Mathur, "WADI: A water distribution testbed for research in the design of secure cyber physical systems," in *Proc. 3rd Int. Workshop Cyber-Phys. Syst. Smart Water Netw.*, 2017, pp. 25–28.

- [65] A. Abdulaal, Z. Liu, and T. Lancewicki, “Practical approach to asynchronous multivariate time series anomaly detection and localization,” in *Proc. 27th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2021, pp. 2485–2494.



Qihang Zhou (Student Member, IEEE) received the BS degree from the China University of Geosciences, Wuhan, China, in 2020. He is currently working toward the PhD degree in control science and engineering with the College of Control Science and Engineering, Zhejiang University, Hangzhou, China. His research interests include generative learning, data mining, and anomaly detection.



Shibo He (Senior Member, IEEE) received the PhD degree in control science and engineering from Zhejiang University, Hangzhou, China, in 2012. He is currently a professor with Zhejiang University. He was an associate research scientist from March 2014 to May 2014, and a postdoctoral scholar from 2012 to 2014, with Arizona State University, Tempe, AZ, USA. From 2010 to 2011, he was a visiting scholar with the University of Waterloo, Waterloo, ON, Canada. His research interests include Internet of Things, crowdsensing, Big Data analysis, etc. He serves on

the editorial board of *IEEE Transactions on Vehicular Technology*, *Springer Peer-to-Peer Networking and Application* and *KSII Transactions on Internet and Information Systems*, and is a guest editor of the *Elsevier Computer Communications* and *Hindawi International Journal of Distributed Sensor Networks*. He was a general co-chair for iSCI 2022, symposium co-chair for the IEEE/CIC ICC 2022, IEEE GlobeCom 2020 and the IEEE ICC 2017, TPC co-chair for i-Span 2018, a Finance and Registration chair for ACM MobiHoc 2015, a TPC co-chair for the IEEE ScalCom 2014, a TPC vice co-chair for ANT 2013 and 2014, a track co-chair for the Pervasive Algorithms, Protocols, and Networks of EUSPN 2013, a web co-chair for the IEEE MASS 2013, and a Publicity co-chair of IEEE WiSARN 2010, and FCN 2014.



Haoyu Liu received the PhD degree from Zhejiang University, Hangzhou, China, in 2021. He is currently working with NetEase Fuxi AI Lab, Hangzhou. His research interests include data mining and machine learning, with particular interests in anomaly detection and crowdsourcing.



Jiming Chen (Fellow, IEEE) received the PhD degree in control science and engineering from Zhejiang University, Hangzhou, China, in 2005. He is currently a professor with the Department of Control Science and Engineering, the vice dean with the Faculty of Information Technology, Zhejiang University. His research interests include IoT, networked control, wireless networks. He serves on the editorial boards of multiple IEEE Transactions, and the general co-chairs for IEEE RTCSA'19, IEEE Datacom'19, and IEEE PST'20. He was a recipient of the 7th IEEE ComSoc Asia/Pacific Outstanding Paper Award, the JSPS Invitation Fellowship, and the IEEE ComSoc AP Outstanding Young Researcher Award. He is an IEEE VTS distinguished lecturer. He is a fellow of the CAA.



Wenchao Meng (Senior Member, IEEE) received the PhD degree in control science and engineering from Zhejiang University, Hangzhou, China, in 2015, where he is currently with the College of Control Science and Engineering. His current research interests include adaptive intelligent control, cyber-physical systems, renewable energy systems, and smart grids.